

Implementation of a CAN/CAN-FD Bus Transmitter in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

1	Design and Architecture	1
1.1	System Overview	1
1.1.1	LLC Frame Format	1
1.2	Interface Definition Tables	3
1.3	Protocol-Driven Type System	10
1.4	LLC Sub-layer	10
1.4.1	can_llc_tx	10
1.4.2	can_llc_rx	10
1.5	MAC Sub-layer	10
1.5.1	can_mac_tx	10
1.5.2	can_mac_rx	10
1.6	PCS Sub-layer	10
1.6.1	can_pcs_tx	10
1.6.2	can_pcs_rx	10

1 Design and Architecture

1.1 System Overview

A complete CAN node is composed of a Transmit (TX) path and a Receive (RX) path coordinated by shared fault-confinement and physical-interface control. The paths operate in parallel at runtime, but they are not symmetric in behavior or responsibility. The focus of this thesis is the complete node architecture and its standards-traceable interfaces. The TX path is responsible for frame submission, arbitration participation, and bitstream generation toward the bus, while the RX path is responsible for bus observation, frame reconstruction, and delivery/notification toward the user layer. As shown in Figure 1, this full-node decomposition spans LLC, MAC, PCS, FCE, and PMA boundaries.

1.1.1 LLC Frame Format

The current LLC Frame format is depicted in Figure 2 with the ID byte format depicted in Table 1. The revised LLC Frame supporting FD is depicted in Figure 3. The revised format adds a 3-bit FMT (ISO 6.4.3) field to the DLC byte, expands the data field to 64 bytes, and repurposes reserved bits in the last byte for BRS and ESI flags. The FMT encodes the supported frame formats as ‘000’ = CB, ‘100’ = CE, ‘010’ = FB, ‘110’ = FE. Accordingly, for the frame content to be self-consistent the IDE bit must be set to 0 for FMT = CB/FB and 1 for FMT = CE/FE. The revised format is designed to be backward compatible: an implementation that only supports Classic frames can simply ignore the FMT bits and treat all frames as Classic, while an implementation that supports FD can use the FMT field to distinguish frame types without affecting the

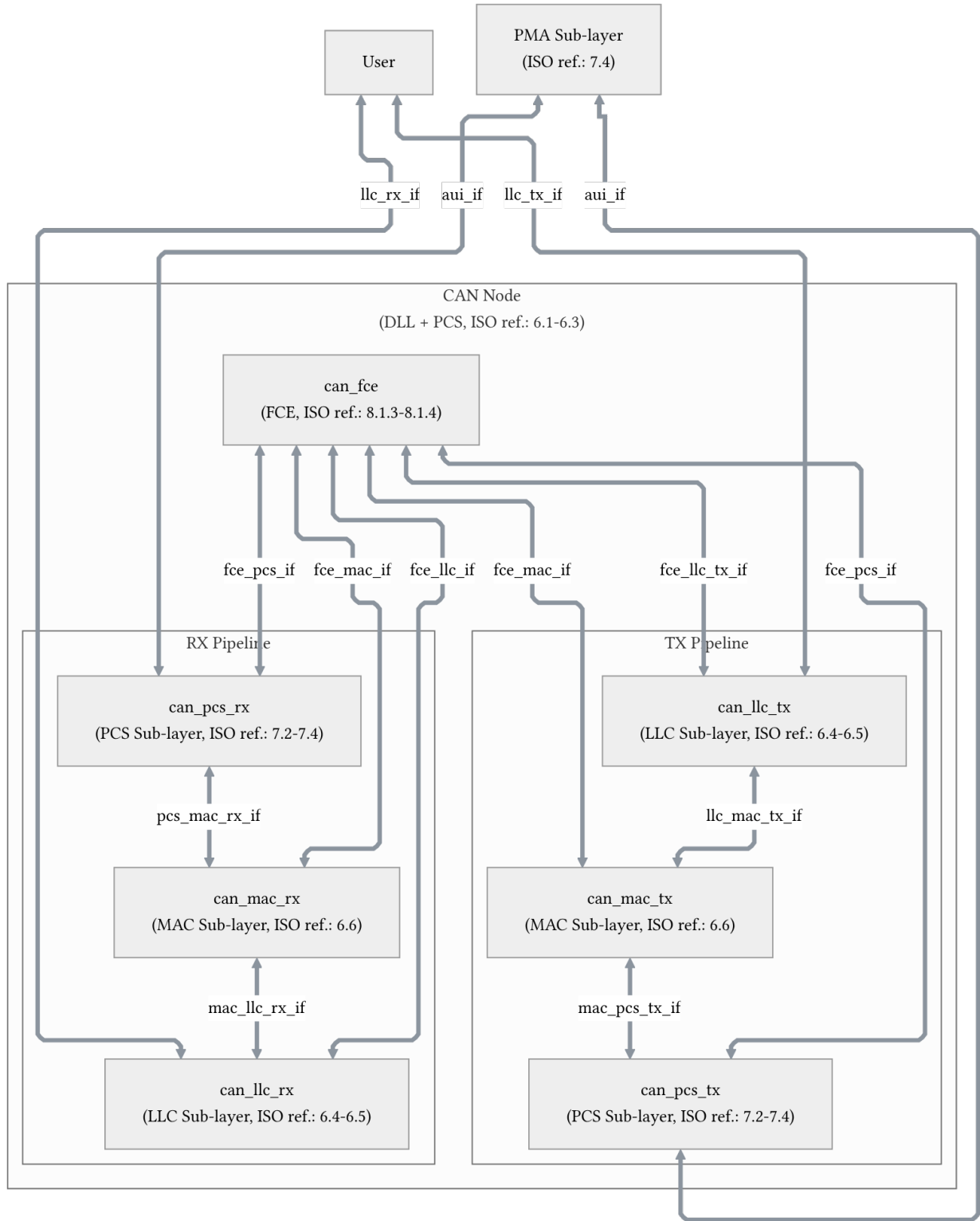


Figure 1: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries. Interface definitions are provided for llc_tx_if (Table 2), llc_rx_if (Table 3), llc_mac_tx_if (Table 4), mac_llc_rx_if (Table 5), mac_pcs_tx_if (Table 6), pcs_mac_rx_if (Table 7), auif (Table 8), fce_llc_tx_if and fce_llc_if (Table 9), fce_mac_if (Table 10), and fce_pcs_if (Table 11).

Table 2: Interface definition for `llc_tx_if`. Uses the existing Avalon-ST interface (`pk_eth_st`) to implement `L_Data.Request`. Includes `L_Data.AbortRequest` for complete ISO-defined service coverage. `L_Data.AbortRequest` is not implemented in the current CAN bus controller. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
<code>L_Data.Request (start)</code>	LLC Frame, Handle	User -> LLC	Request data transfer of LLC frame	Issued when User wants to start LLC frame transfer	6.4.5.5.2	valid && ready && sop && payload on data
<code>L_Data.Request (stream)</code>	LLC Frame, Handle	User -> LLC	Stream LLC frame	Intermediate beats while LLC frame is streamed	6.4.5.5.2	valid && ready && !sop && !eop && payload on data
<code>L_Data.Request (end)</code>	LLC Frame, Handle	User -> LLC	End of current LLC frame transfer	Final beat of LLC frame transfer	6.4.5.5.2	valid && ready && eop && payload on data
<code>L_Data.AbortRequest</code>	Handle	User -> LLC	Abort pending LLC frame transfer	Issued when User wants to cancel LLC frame transfer	6.4.5.5.3	TBD

Table 3: Interface definition for `llc_rx_if`. Uses the existing Avalon-ST interface (`pk_eth_st`) to implement `L_Data.Indication`. Includes `L_Data.Confirm` and optional Timestamp for complete ISO service coverage. `L_Data.AbortRequest` is not implemented in the current CAN bus controller. Transfer_Status: [Ongoing, Lost Arbitration, Transmitted, Aborted, Disturbed]. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
<code>L_Data.Indication (start)</code>	LLC Frame, optional Timestamp	LLC -> User	Start received LLC frame transfer	Issued when LLC wants to start LLC frame transfer	6.4.5.5.5	valid && ready && sop && payload on data
<code>L_Data.Indication (stream)</code>	LLC Frame, optional Timestamp	LLC -> User	Stream received LLC frame	Intermediate beats while LLC frame is streamed	6.4.5.5.5	valid && ready && !sop && !eop && payload on data

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
L_Data.Indication (end)	LLC Frame, optional Timestamp	LLC -> User	End received LLC frame transfer	Final beat of LLC frame transfer	6.4.5.5.5	valid && ready && eop && payload on data
L_Data.Confirm	Transfer_Status, optional Timestamp, Handle	LLC -> User	Report result of prior L_Data.Request	Issued on completion/failure of prior L_Data.Request	6.4.5.5.4	TBD

Table 4: Interface definition for llc_mac_tx_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
DLL SDU	LLC Frame	LLC -> MAC	Transfer LLC Frame content	Issued when LLC starts MAC handoff	6.3, 6.6.4.2	TBD
Interface control information	Control context	LLC -> MAC	Transfer control context for frame handling	Valid with DLL SDU handoff	6.6.4.2	TBD

Table 5: Interface definition for mac_llc_rx_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
DLL SDU	Reconstructed LLC Frame	MAC -> LLC	Transfer reconstructed LLC Frame	Issued when MAC has reconstructed LLC Frame from received MAC Frame	6.6.4.3, 6.6.9	TBD
Time reference notification	SOF/frame-valid indication	MAC -> LLC	Provide timing reference for timestamping	Generated for transmitted/received DF/RF	6.6.3	TBD

Table 6: Interface definition for mac_pcs_tx_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
PCS_Data.Request	Output_Unit	MAC -> PCS	Request transmission of one dominant/recessive bit	Bit-by-bit during MAC Frame serialization	7.2.1, 7.2.2	TBD
PCS_Status.Transmitter	D_Transmit	MAC -> PCS	Indicate FD data-phase interval	Active during transmission in the FD data-phase	7.2.1, 7.2.5	TBD

Table 7: Interface definition for pcs_mac_rx_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
PCS_Data.Indicate	Input_Unit	PCS -> MAC	Indicate arrival of one dominant/recessive bit	Bit-by-bit during reception stream	7.2.1, 7.2.3	TBD
PCS_Status.Receiver	D_Receive	MAC -> PCS	Indicate FD data-phase interval	Active during reception in the FD data-phase	7.2.1, 7.2.6	TBD

Table 8: Interface definition for aui_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
output symbol	Dominant/recessive symbol	PCS -> PMA	Drive physical output symbol	Issued on Output_Unit updates	7.4.2.1	TBD
bus_off symbol	Bus-off control	PCS -> PMA	Switch node off bus	Issued on Bus_off_request request from FCE	7.4.2.2, 8.1.3.4	TBD

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
bus_off_release symbol	Bus-off release control	PCS -> PMA	Release node from bus-off	Issued on Bus_off_release request from FCE	7.4.2.3, 8.1.3.4	TBD
input symbol	Dominant/recessive symbol	PMA -> PCS	Indicate physical input symbol	Continuous PMA-to-PCS indication	7.4.3	TBD

Table 9: Interface definition for fce_llc_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Normal_mode_request	Mode request	LLC -> FCE	Request reset to normal mode	Issued on startup/restart	8.1.3.2	TBD
Normal_mode_response	Mode response	FCE -> LLC	Acknowledge normal-mode request	Returned after FCE processing	8.1.3.2	TBD
Bus_off	Bus-off status	FCE -> LLC	Indicate node is bus-off	Asserted on bus-off transition	8.1.3.2	TBD

Table 10: Interface definition for fce_mac_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Transmit/receive	Transfer mode context	MAC -> FCE	Report current TX/RX context	Updated with MAC transfer context	8.1.3.3	TBD
Error	Error event	MAC -> FCE	Report detected protocol error	On bit/stuff/CRC/form/ACK error	8.1.3.3	TBD

∞

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Primary_error	Primary error event	MAC -> FCE	Report primary error condition	On primary error condition	8.1.3.3	TBD
Error/overload flag	EF/OF state	MAC -> FCE	Report EF/OF transmission state	During EF/OF transmission	8.1.3.3	TBD
Counters_unchanged	Counter-update qualifier	MAC -> FCE	Qualify counter exception path	On rule-c exception cases	8.1.3.3	TBD
Error_delimiter_too_late	Late delimiter event	MAC -> FCE	Report late error-delimiter condition	Set on late delimiter condition	8.1.3.3	TBD
Successful_transfer	Transfer completion event	MAC -> FCE	Report successful TX/RX completion	On successful frame transfer	8.1.3.3	TBD
Error_passive_response	State response	MAC -> FCE	Report entry into error-passive state	On state transition completion	8.1.3.3	TBD
Error_active_response	State response	MAC -> FCE	Report return to error-active state	On state transition completion	8.1.3.3	TBD
Error_passive_request	State request	FCE -> MAC	Request MAC enter error-passive state	On TEC/REC threshold crossing	8.1.3.3	TBD
Error_active_request	State request	FCE -> MAC	Request MAC return to error-active state	On TEC/REC recovery conditions	8.1.3.3	TBD

Table 11: Interface definition for fce_pcs_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Bus_off_request	Bus-off request	FCE -> PCS	Request node switch-off from bus	On bus-off transition condition	8.1.3.4	TBD

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Bus_off_release_request	Bus-off release request	FCE -> PCS	Request node re-enable from bus-off	On restart/reintegration	8.1.3.4	TBD
Bus_off_response	Bus-off response	PCS -> FCE	Acknowledge bus-off request	Returned after bus-off action	8.1.3.4	TBD
Bus_off_release_response	Bus-off release response	PCS -> FCE	Acknowledge bus-off-release request	Returned after release action	8.1.3.4	TBD

1.3 Protocol-Driven Type System

1.4 LLC Sub-layer

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

1.4.1 `can_llc_tx`

1.4.2 `can_llc_rx`

1.5 MAC Sub-layer

The core of the protocol logic. It handles bit serialization, CRC calculation, and bit stuffing. It coordinates the overall frame state machine and interacts closely with the Fault Confinement Entity (FCE) to manage error counters (TEC/REC) and node state (Error Active/Passive/Bus Off) based on protocol violations.

1.5.1 `can_mac_tx`

The MAC TX layer (Figure 5) is composed of four main components:

1. `tx_mac_ser` (Serializer): Converts LLC bytes into a serial bit stream with polarity information
2. `tx_mac_fsm` (Frame State Machine): Coordinates frame transmission, controls all submodules
3. `bit_stuffer_fd` (Bit Stuffer): Implements CAN/CAN-FD bit stuffing rules
4. `crc_fd` (CRC Engine): Generates CRC-15/CRC-17/CRC-21 based on frame type

1.5.2 `can_mac_rx`

1.6 PCS Sub-layer

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.

1.6.1 `can_pcs_tx`

1.6.2 `can_pcs_rx`

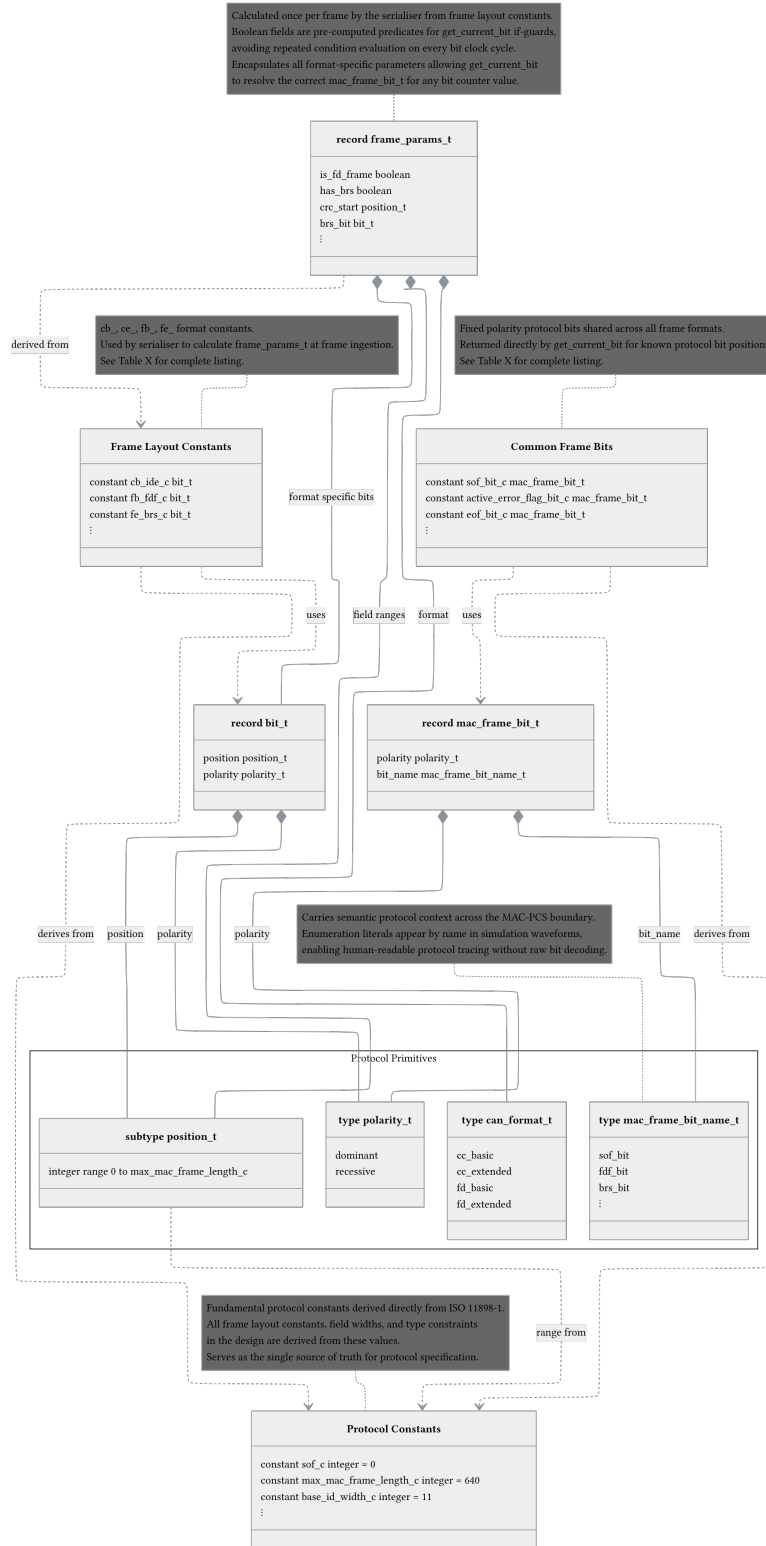


Figure 4: Type and constant hierarchy. Protocol constants derived from ISO 11898-1 form the root of the hierarchy, from which frame layout constants, type constraints, and composite record types are derived. The mac_frame_bit_name_t enumeration carries semantic protocol context across the MAC-PCS boundary and provides human-readable signal names in simulation waveforms.

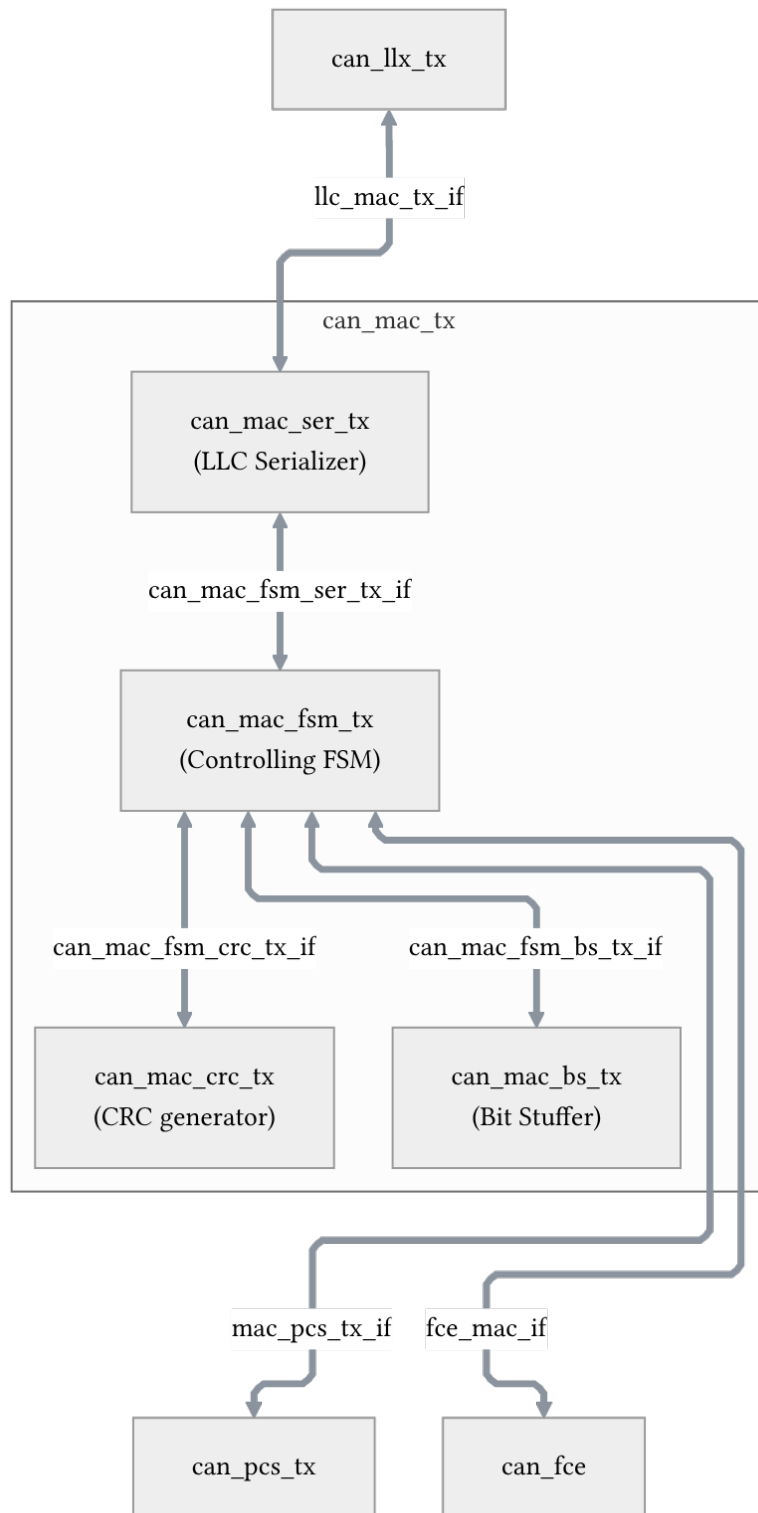


Figure 5: MAC TX layer architecture showing internal component interconnections and external interfaces (LLC, PCS, FCE).